SWE 261P Software Testing and Analysis - Part 5 Report

PDFsam Basic: Testable Design and Mocking

JAVA 21

JUNIT 5

MAVEN

MOCKITO 5

Course SWE 261P
Quarter Winter 2026
Team Kingson & Zian & Zhenyu

Repo Github Link:
https://github.com/eric-song-dev/pdfsam

Team Members:

- Kingson Zhang: kxzhang@uci.edu
- Zian Xu: zianx11@uci.edu
- Zhenyu Song: zhenyus4@uci.edu

This report documents our analysis of **testable design** principles and application of **stub** and **mock** testing techniques to **PDFsam Basic**.

📂 Quick Navigation

- ▼ SWE 261P Software Testing and Analysis - Part 5 Report
  - PDFsam Basic: Testable Design and Mocking
  - 📂 Quick Navigation
  - ▼ 🎯 1. Testable Design: Principles and Goals
    - 1.1 What Is Testable Design?
    - 1.2 Key Principles
    - 1.3 Why Testable Design Matters
    - 1.4 Test Doubles Overview
  - ▼ 📦 2. Testable Design in PDFsam
    - 2.1 Good Design: Interface-Based Services
    - 2.2 Bad Design Patterns Found
    - 2.3 Good Design: Constructor Injection
  - ▼ 🔧 3. Zhenyu's Stubbing: UsageService
    - 3.1 Existing Stub Found
    - 3.2 New Stub Implementation
    - 3.3 Stub Test Cases
  - ▼ 🔧 4. Kingson's Stubbing: NewsService
    - 4.1 Existing Stub Found
    - 4.2 New Stub Implementation
    - 4.3 Stub Test Cases
  - ▼ 🔧 5. Zian's Stubbing: UpdateService
    - 5.1 Existing Stub Found
    - 5.2 New Stub Implementation
    - 5.3 Stub Test Cases
  - ▼ ⚠️ 6. Zhenyu's Bad Testable Design
    - 6.1 Problem Identification
    - 6.2 Proposed Fix: Injectable StatusUpdater + PdfParserFunction
    - 6.3 Test Cases for the Fix
  - ▼ ⚠️ 7. Kingson's Bad Testable Design
    - 7.1 Problem Identification
    - 7.2 Proposed Fix: Inject ExecutorService via Constructor
    - 7.3 SynchronousExecutorService for Deterministic Tests
    - 7.4 Test Cases for the Fix
  - ▼ ⚠️ 8. Zian's Bad Testable Design
    - 8.1 Problem Identification

- 8.2 Proposed Fix: Injectable VersionDataProvider
- 8.3 Test Cases for the Fix

▼ 🔬 9. Mocking with Mockito

- 9.1 What Is Mocking?
- 9.2 Mockito Framework
- 9.3 Why Mocking Is Useful

▼ 🔬 10. Zhenyu's Mocking: TaskExecutionService

- 10.1 Feature Mocked
- 10.2 Mock Test Cases

▼ 🔬 11. Kingson's Mocking

- 11.1 Feature Mocked
- 11.2 Mock Test Cases

▼ 🔬 12. Zian's Mocking

- 12.1 Feature Mocked
- 12.2 Mock Test Cases

▼ 📋 13. Test Implementation Summary

- 13.1 New Test Files
- 13.2 Test Breakdown
- 13.3 Running the Tests
- 13.4 Test Results

▼ 🎯 14. Conclusion

- Key Takeaways

🎯 1. Testable Design: Principles and Goals

1.1 What Is Testable Design?

Testable design is a set of architectural principles that make code easy to test in isolation. The goal is to ensure that each unit of code can be exercised by automated tests without requiring complex setup, external systems, or the full application runtime.

1.2 Key Principles

Principle	Description	PDFsam Example	Violation Impact
Dependency Injection	Pass dependencies via constructors or setters rather than creating them internally with `new`	`TaskExecutionController` receives `TaskExecutionService` and `UsageService` via `@Inject` constructor — easily replaceable in tests	Dependencies cannot be replaced with test doubles
Avoid Static Methods	Use instance methods that can be overridden or mocked	⚠️ `DefaultPdfLoadService.fxMoveStatusTo()` is `private static` and calls `Platform.runLater()` — **untestable** without JavaFX runtime (see Section 6)	Static methods with side effects cannot be stubbed
Avoid Logic in Constructors	Keep constructors simple; place complex logic in overridable methods	`DefaultRecentWorkspacesService` reads from `PreferencesRepository` inside its constructor, coupling initialization to persistence	Subclass constructors cannot bypass superclass logic
Avoid Complex Private Methods	Keep private methods simple; extract complex logic	`DefaultPdfLoadService.accept()` contains multi-branch loading logic delegated through private helpers	Hidden bugs in untested private logic

Principle	Description	PDFsam Example	Violation Impact
	into testable units		
Avoid Singleton Pattern	Ensure classes that need swapping during tests are not singletons	PDFsam uses Jakarta CDI scoping instead of manual singletons, keeping classes injectable	Cannot replace singleton instances for testing
Program to Interfaces	Depend on abstractions (interfaces) rather than concrete implementations	All services (`NewsService` , `UpdateService` , `UsageService` , etc.) are defined as interfaces with `Default*` implementations	Cannot substitute test doubles for real objects

1.3 Why Testable Design Matters

A well-designed testable system enables:

- **Isolation**: Test individual components without their dependencies
- **Speed**: Tests run fast without I/O, network, or database access
- **Determinism**: Controlled inputs produce predictable outputs
- **Flexibility**: Swap real dependencies with stubs, mocks, or spies
- **Maintainability**: Tests serve as documentation and catch regressions early

1.4 Test Doubles Overview

Type	Purpose	Returns Values?	Verifies Behavior?
Stub	Provides canned answers to support the test	✅	❌
Mock	Verifies how it was used (method calls, arguments)	Optional	✅
Spy	Wraps a real object and records interactions	✅ (real)	✅

📦 2. Testable Design in PDFsam

PDFsam Basic already follows several testable design principles, but also has areas that could be improved:

2.1 Good Design: Interface-Based Services

PDFsam uses the **Strategy Pattern / interface-based design** extensively in its service layer:

Interface	Implementation	Purpose	Testability Benefit
`NewsService`	`DefaultNewsService`	News fetching from remote JSON	Tests can stub to avoid network I/O
`UpdateService`	`DefaultUpdateService`	Version checking via remote URL	Tests can stub to return fixed versions
`UsageService`	`DefaultUsageService`	Tool usage tracking with Preferences API	Tests can stub to record calls in-memory
`StageService`	`DefaultStageService`	Window state management	Tests can mock to verify save/clear calls
`RecentWorkspacesService`	`DefaultRecentWorkspacesService`	Workspace history persistence	Tests can mock the underlying repository

Interface	Implementation	Purpose	Testability Benefit
`PdfLoadService`	`DefaultPdfLoadService`	PDF document loading	Tests can stub to avoid actual PDF parsing

This design allows any consumer of these services to depend on the *interface*, enabling easy substitution with stubs or mocks during testing. For example, `TaskExecutionController` depends on `UsageService` (interface), so in our stubbing tests we replaced it with a `StubUsageService` without changing the controller code.

2.2 Bad Design Patterns Found

While the interface layer is well designed, we identified several **violations** within the implementations:

Location	Issue	Why It Hurts Testability
`DefaultPdfLoadService.fxMoveStatusTo()`	`private static` + `Platform.runLater()`	Cannot unit-test status transitions without JavaFX runtime
`TaskExecutionController`	`ExecutorService` created as field initializer	Cannot inject a synchronous executor for deterministic tests
`DefaultUpdateService.getLatestVersion()`	`new URL(...)` hardcoded inside method	Cannot intercept data source; requires real file/network

These issues are analyzed in detail in Sections 6–8, with proposed fixes and test cases.

2.3 Good Design: Constructor Injection

Services use Jakarta `@Inject` for dependency injection:

```
@Inject
public TaskExecutionController(TaskExecutionService executionService, UsageService usageS
    this.executionService = executionService;
    this.usageService = usageService;
    // ...
}
```

This makes it straightforward to pass mock or stub implementations during testing. In our tests, we directly instantiate controllers with test doubles:

```
// Stub injection — no DI framework needed in tests
StubUsageService stub = new StubUsageService();
TaskExecutionController controller = new TaskExecutionController(mockExecutionService, stu
```

🔧 3. Zhenyu's Stubbing: UsageService

Test File: ZhenyuPart5Test.java

3.1 Existing Stub Found

Location: StageServiceControllerTest.java

In `StageServiceControllerTest` , the `StageService` *interface* is used as a **stub via Mockito** to isolate the controller. While Mockito is often associated with mocking, here it acts in a *stub role* — providing canned responses without verifying interactions:

```
@BeforeEach
public void setUp() {
    this.service = mock(StageService.class);  // Mockito mock acting as a stub
    this.victim = new StageServiceController(service);
}
```

Why it is used: `StageServiceController` depends on `StageService` to persist stage status. By stubbing the interface, the test verifies the controller's event-handling logic without touching the real `DefaultEntityRepository` or the Java Preferences API. The key point is that `StageService` is an *interface*, which enables easy substitution — a hallmark of testable design.

3.2 New Stub Implementation

A **hand-written** `StubUsageService` that implements the `UsageService` interface, recording calls in memory. Unlike a Mockito-generated stub, this is a real class with actual logic, demonstrating the *manual stub* pattern:

```java
static class StubUsageService implements UsageService {
    private final List<String> incrementedModules = new ArrayList<>();
    private long totalUsages = 0;

    @Override
    public void incrementUsageFor(String moduleId) {
        incrementedModules.add(moduleId);
        totalUsages++;
    }

    @Override public void clear() { incrementedModules.clear(); totalUsages = 0; }
    @Override public long getTotalUsages() { return totalUsages; }
    public List<String> getIncrementedModules() { return incrementedModules; }
}
```

3.3 Stub Test Cases

```java
@Test
void stubRecordsUsageIncrement() {
    AbstractParameters params = mock(AbstractParameters.class);
    TaskExecutionRequest request = new TaskExecutionRequest("merge", params);
    controller.request(request);

    assertEquals(1, stubUsageService.getTotalUsages());
    assertEquals("merge", stubUsageService.getIncrementedModules().get(0));
}

@Test
void stubRecordsMultipleIncrements() {
    AbstractParameters params = mock(AbstractParameters.class);
    controller.request(new TaskExecutionRequest("merge", params));
    controller.request(new TaskExecutionRequest("split", params));
    controller.request(new TaskExecutionRequest("merge", params));

    assertEquals(3, stubUsageService.getTotalUsages());
    assertEquals(List.of("merge", "split", "merge"), stubUsageService.getIncrementedModul
}
```

🔧 4. Kingson's Stubbing: NewsService

Test File: KingsonPart5Test.java

4.1 Existing Stub Found

Location: DefaultNewsServiceTest.java

In `DefaultNewsServiceTest` , `PreferencesRepository` is mocked as a stub to provide controlled responses:

```
repo = mock(PreferencesRepository.class);
victim = new DefaultNewsService(appBrand, mapper, repo);

// Used as a stub — returns canned value
when(repo.getInt(LATEST_NEWS_ID, -1)).thenReturn(5);
```

Why it is used: `DefaultNewsService` reads/writes to the Java Preferences API. The mock provides predictable values so the test can verify the service's behavior around exception handling and data persistence without a real preferences store.

4.2 New Stub Implementation

The existing consumer of `NewsService` in PDFsam is `NewsController` , but its `fetchLatestNews()` method launches a virtual thread and broadcasts results via `eventStudio()` , making it difficult to test synchronously with a simple stub. Therefore, we created a lightweight `NewsChecker` helper that consumes `NewsService` synchronously, allowing us to clearly demonstrate the stub pattern.

A manual `StubNewsService` returning hardcoded news data without network access:

```java
static class StubNewsService implements NewsService {
    private final List<NewsData> stubbedNews;
    private int latestNewsSeen = -1;
    private int latestImportantNewsSeen = -1;

    StubNewsService(List<NewsData> news) { this.stubbedNews = news; }

    @Override public List<NewsData> getLatestNews() { return stubbedNews; }
    @Override public int getLatestNewsSeen() { return latestNewsSeen; }
    @Override public void setLatestNewsSeen(int id) { this.latestNewsSeen = id; }
    // ...
}
```

4.3 Stub Test Cases

```java
@Test
void stubReturnsUnseenImportantNews() {
    List<NewsData> news = List.of(
        new NewsData(1, "Title 1", "Content 1", LocalDate.now(), "http://link1.com", true
    StubNewsService stub = new StubNewsService(news);
    NewsChecker checker = new NewsChecker(stub);

    assertTrue(checker.hasUnseenImportantNews());
    assertEquals(1, checker.getNewsCount());
}

@Test
void stubAfterMarkAllSeen() {
    // After marking all as seen, no unseen important news
    StubNewsService stub = new StubNewsService(List.of(
        new NewsData(10, "Important", "Content", LocalDate.now(), "http://link.com", true
    NewsChecker checker = new NewsChecker(stub);
    checker.markAllSeen();

    assertFalse(checker.hasUnseenImportantNews());
}
```

🔧 5. Zian's Stubbing: UpdateService

Test File: ZianPart5Test.java

5.1 Existing Stub Found

Location: DefaultUpdateServiceTest.java

In `DefaultUpdateServiceTest` , `AppBrand` is mocked as a stub to return a local file URL instead of a remote server:

```
@BeforeEach
public void setUp() {
    appBrand = mock(AppBrand.class);
    this.mapper = JsonMapper.builder().addModule(new Jdk8Module()).addModule(new JavaTimeI
            .enable(DeserializationFeature.FAIL_ON_NUMBERS_FOR_ENUMS).enable(Serializatio
            .disable(SerializationFeature.WRITE_DATE_TIMESTAMPS_AS_NANOSECONDS)
            .visibility(PropertyAccessor.FIELD, JsonAutoDetect.Visibility.ANY)
            .serializationInclusion(JsonInclude.Include.NON_EMPTY)
            .build();
    victim = new DefaultUpdateService(appBrand, mapper);
}

// Stub provides a local file URL instead of real server
public void pasitiveCheckForUpdates(@TempDir Path folder) throws IOException {
    var file = Files.createTempFile(folder, null, null);
    Files.copy(getClass().getResourceAsStream("/test_current_version.json"), file,
            StandardCopyOption.REPLACE_EXISTING);
    when(appBrand.property(BrandableProperty.CURRENT_VERSION_URL)).thenReturn(file.toFile
    assertEquals("3.0.0", victim.getLatestVersion());
}
```

Why it is used: `DefaultUpdateService.getLatestVersion()` fetches version info from a remote URL. By stubbing `AppBrand` to return a local file path, the test avoids network dependency while still exercising the JSON parsing logic.

5.2 New Stub Implementation

The existing consumer of `UpdateService` in PDFsam is `UpdatesController` , but its `checkForUpdates()` method also uses a virtual thread with `eventStudio().broadcast()` , making

synchronous stub testing impractical. We created a simple `UpdateChecker` wrapper to demonstrate the stub pattern cleanly.

A manual `StubUpdateService` returning a fixed version string:

```
static class StubUpdateService implements UpdateService {
    private final String version;
    StubUpdateService(String version) { this.version = version; }

    @Override
    public String getLatestVersion() { return version; }
}
```

5.3 Stub Test Cases

```
@Test
void stubNewerVersion() {
    StubUpdateService stub = new StubUpdateService("6.0.0");
    UpdateChecker checker = new UpdateChecker(stub, "5.4.5");

    assertTrue(checker.isUpdateAvailable());
    assertEquals("Update available: 6.0.0", checker.getUpdateMessage());
}

@Test
void stubSameVersion() {
    StubUpdateService stub = new StubUpdateService("5.4.5");
    UpdateChecker checker = new UpdateChecker(stub, "5.4.5");

    assertFalse(checker.isUpdateAvailable());
    assertEquals("You are using the latest version.", checker.getUpdateMessage());
}
```

⚠️ 6. Zhenyu's Bad Testable Design

6.1 Problem Identification

Location: DefaultPdfLoadService.java (Line 99)

```
private static void fxMoveStatusTo(PdfDocumentDescriptor descriptor, PdfDescriptorLoading
    Platform.runLater(() -> descriptor.moveStatusTo(status));
}
```

Why this is bad testable design:

Issue	Impact
`private static` method	Cannot be overridden or stubbed in a subclass
Calls `Platform.runLater()`	Requires a running JavaFX Application Thread
Tightly couples loading logic to JavaFX runtime	Cannot unit-test the PDF loading state transitions without starting the entire application

6.2 Proposed Fix: Injectable StatusUpdater + PdfParserFunction

Extract the status-updating behavior into a pluggable interface, and make the PDF parsing step injectable:

```java
interface StatusUpdater {
    void moveStatusTo(PdfDocumentDescriptor descriptor, PdfDescriptorLoadingStatus status
}

// Production: uses Platform.runLater()
class FxStatusUpdater implements StatusUpdater { ... }

// Test: updates directly without JavaFX
class DirectStatusUpdater implements StatusUpdater {
    @Override
    public void moveStatusTo(PdfDocumentDescriptor descriptor, PdfDescriptorLoadingStatus
        // Directly update without Platform.runLater()
        descriptor.moveStatusTo(status);
        statusLog.add(descriptor.getFileName() + " -> " + status.name());
    }
}

// Injectable parser function — replaces hardcoded PDFParser.parse()
@FunctionalInterface
interface PdfParserFunction {
    void parse(PdfDocumentDescriptor descriptor) throws Exception;
}
```

The `TestablePdfLoadLogic` covers all **4 branches** from the original `DefaultPdfLoadService.load()` :

Branch	Condition	Status Transition
1	No password, parse succeeds	LOADING → LOADED
2	Has password, parse succeeds	LOADING → LOADED_WITH_USER_PWD_DECRYPTION
3	InvalidPasswordException	LOADING → ENCRYPTED
4	General Exception	LOADING → WITH_ERRORS

6.3 Test Cases for the Fix

```
@Test
@DisplayName("Successful load without password transitions LOADING → LOADED")
void loadWithoutPassword() {
    File tempFile = new File("test.pdf");
    PdfDocumentDescriptor descriptor = PdfDocumentDescriptor.newDescriptorNoPassword(templ
    descriptor.moveStatusTo(PdfDescriptorLoadingStatus.REQUESTED);

    loadLogic.loadDescriptor(descriptor);

    assertEquals(PdfDescriptorLoadingStatus.LOADED, descriptor.loadingStatus().getValue()
}

@Test
@DisplayName("Successful load with password transitions LOADING → LOADED_WITH_USER_PWD_DE
void loadWithPassword() {
    File tempFile = new File("encrypted.pdf");
    PdfDocumentDescriptor descriptor = PdfDocumentDescriptor.newDescriptor(tempFile, "sec
    descriptor.moveStatusTo(PdfDescriptorLoadingStatus.REQUESTED);

    loadLogic.loadDescriptor(descriptor);

    assertEquals(PdfDescriptorLoadingStatus.LOADED_WITH_USER_PWD_DECRYPTION, descriptor.l
}

@Test
@DisplayName("Invalid password transitions LOADING → ENCRYPTED")
void loadWithInvalidPassword() {
    File tempFile = new File("locked.pdf");
    PdfDocumentDescriptor descriptor = PdfDocumentDescriptor.newDescriptorNoPassword(templ
    descriptor.moveStatusTo(PdfDescriptorLoadingStatus.REQUESTED);

    // Inject a parser that throws InvalidPasswordException
    TestablePdfLoadLogic errorLogic = new TestablePdfLoadLogic(statusUpdater,
            d -> {
                throw new InvalidPasswordException("Bad password");
            });

    errorLogic.loadDescriptor(descriptor);

    assertEquals(PdfDescriptorLoadingStatus.ENCRYPTED, descriptor.loadingStatus().getValu
}
```

```java
@Test
@DisplayName("General exception during load transitions LOADING → WITH_ERRORS")
void loadWithGeneralError() {
    File tempFile = new File("corrupted.pdf");
    PdfDocumentDescriptor descriptor = PdfDocumentDescriptor.newDescriptorNoPassword(templ
    descriptor.moveStatusTo(PdfDescriptorLoadingStatus.REQUESTED);

    // Inject a parser that throws a general exception
    TestablePdfLoadLogic errorLogic = new TestablePdfLoadLogic(statusUpdater,
            d -> {
                throw new RuntimeException("Corrupt PDF");
            });

    errorLogic.loadDescriptor(descriptor);

    assertEquals(PdfDescriptorLoadingStatus.WITH_ERRORS, descriptor.loadingStatus().getVa
}

@Test
@DisplayName("StatusUpdater logs all transitions correctly")
void statusUpdaterLogs() {
    File tempFile = new File("test.pdf");
    PdfDocumentDescriptor descriptor = PdfDocumentDescriptor.newDescriptorNoPassword(templ
    descriptor.moveStatusTo(PdfDescriptorLoadingStatus.REQUESTED);

    loadLogic.loadDescriptor(descriptor);

    assertEquals(2, statusUpdater.getStatusLog().size());
    assertTrue(statusUpdater.getStatusLog().get(0).contains("LOADING"));
    assertTrue(statusUpdater.getStatusLog().get(1).contains("LOADED"));
}
```

⚠️ 7. Kingson's Bad Testable Design

7.1 Problem Identification

Location: TaskExecutionController.java (Line 57)

```
public class TaskExecutionController {
    private final ExecutorService executor = Executors.newSingleThreadExecutor();
    // ...
    public void request(TaskExecutionRequest event) {
        executor.execute(() -> executionService.execute(event.parameters()));
    }
}
```

Why this is bad testable design:

Issue	Impact
`ExecutorService` created as field initializer	Cannot be replaced via constructor or setter
Asynchronous execution	Tests are non-deterministic — task may not complete before assertion
No shutdown control in tests	Tests cannot verify task ordering or completion timing

7.2 Proposed Fix: Inject ExecutorService via Constructor

```java
static class TestableTaskExecutionController {
    private final TaskExecutionService executionService;
    private final UsageService usageService;
    private final ExecutorService executor;

    // Injectable ExecutorService for testability
    TestableTaskExecutionController(TaskExecutionService executionService,
                                    UsageService usageService,
                                    ExecutorService executor) {
        this.executionService = executionService;
        this.usageService = usageService;
        this.executor = executor;
    }
}
```

7.3 SynchronousExecutorService for Deterministic Tests

```java
static class SynchronousExecutorService extends AbstractExecutorService {
    @Override
    public void execute(Runnable command) { command.run(); } // Runs immediately
    // ...
}
```

7.4 Test Cases for the Fix

```java
@Test
void synchronousExecution() {
    TaskParameters params = mock(TaskParameters.class);
    controller.request(new TaskExecutionRequest("merge", params));

    // With synchronous executor, execution happens immediately — deterministic
    verify(mockExecutionService).execute(params);
    verify(mockUsageService).incrementUsageFor("merge");
}
```

⚠️ 8. Zian's Bad Testable Design

8.1 Problem Identification

Location: DefaultUpdateService.java (Lines 54–65)

```
@Override
public String getLatestVersion() {
    try {
        return objectMapper.readValue(
            new URL(String.format(                              // ← Hardcoded URL creati
                appBrand.property(BrandableProperty.CURRENT_VERSION_URL),
                appBrand.property(BrandableProperty.VERSION))), Map.class)
            .getOrDefault(CURRENT_VERSION_KEY, "").toString();
    } catch (IOException e) {
        LOG.warn(i18n().tr("Unable to find the latest available version."), e);
    }
    return EMPTY;
}
```

Why this is bad testable design:

Issue	Impact
`new URL(...)` hardcoded inside method	Cannot intercept or replace the data source
Reads from network/file URL directly	Tests require a valid file path or network; brittle
Error handling only testable via real I/O failure	Cannot simulate `IOException` in a controlled way

8.2 Proposed Fix: Injectable VersionDataProvider

```java
interface VersionDataProvider {
    InputStream fetchVersionData() throws IOException;
}

static class TestableUpdateService implements UpdateService {
    private final VersionDataProvider dataProvider;
    private final ObjectMapper objectMapper;

    @Override
    public String getLatestVersion() {
        try (InputStream is = dataProvider.fetchVersionData()) {
            Map<String, Object> map = objectMapper.readValue(is, Map.class);
            return map.getOrDefault("currentVersion", "").toString();
        } catch (IOException e) { /* log */ }
        return "";
    }
}
```

8.3 Test Cases for the Fix

```java
@Test
void readVersionFromProvider() {
    VersionDataProvider provider = () ->
        new ByteArrayInputStream("{\"currentVersion\":\"6.0.0\"}".getBytes());
    TestableUpdateService service = new TestableUpdateService(provider, objectMapper);

    assertEquals("6.0.0", service.getLatestVersion());
}

@Test
void returnsEmptyOnError() {
    VersionDataProvider provider = () -> { throw new IOException("Network error"); };
    TestableUpdateService service = new TestableUpdateService(provider, objectMapper);

    assertEquals("", service.getLatestVersion());
}
```

🔬 9. Mocking with Mockito

9.1 What Is Mocking?

Mocking is a testing technique where you create fake objects that simulate the behavior of real objects while also **recording how they are used**. Unlike stubs (which only provide canned return values), mocks allow you to:

- **Verify method calls**: Assert that specific methods were called
- **Verify arguments**: Check that methods received the correct parameters
- **Verify call counts**: Ensure methods were called the expected number of times
- **Verify call order**: Assert the sequence of method invocations

9.2 Mockito Framework

Mockito is Java's most popular mocking framework. Key APIs used in our tests:

API	Purpose
`mock(Class.class)`	Create a mock instance
`when(...).thenReturn(...)`	Configure stub behavior
`verify(mock).method()`	Assert method was called
`verify(mock, times(n))`	Assert method called n times
`verify(mock, never())`	Assert method was never called
`verifyNoMoreInteractions(mock)`	Assert no unexpected calls
`doThrow(...).when(mock)`	Configure exception-throwing behavior
`anyString()` , `eq(...)`	Argument matchers

9.3 Why Mocking Is Useful

Mocking enables **behavior verification** that is not possible with simple assertions:

Without Mocking	With Mocking
Can only check return values and state	Can verify *how* dependencies were used

Without Mocking	With Mocking
Cannot test void methods meaningfully	Can verify void methods were called with correct args
Cannot detect unnecessary side effects	`verifyNoMoreInteractions` catches unexpected calls

🔬 10. Zhenyu's Mocking: TaskExecutionService

Test File: ZhenyuPart5Test.java

10.1 Feature Mocked

`TaskExecutionService.execute()` — when `TaskExecutionController.request()` is called, it should delegate to `executionService.execute(params)` via an internal executor thread.

Why mocking is needed: The `execute()` method runs asynchronously inside a `SingleThreadExecutor`. Without mocking, we cannot verify that the correct parameters were forwarded to the execution service. Mocking gives us:

- **Behavior verification** via `verify()` — confirm `execute()` was called with the exact `AbstractParameters` object
- **Argument capture** via `ArgumentCaptor` — inspect what parameters were passed across multiple calls
- **Exception simulation** via `doThrow()` — verify the controller handles execution failures gracefully

> **Note**: This intentionally mocks a *different* dependency (`TaskExecutionService`) than the stub section (`UsageService`), demonstrating that the same controller can be tested from two angles: state-based (stub) and behavior-based (mock).

10.2 Mock Test Cases

```
@Test
@DisplayName("request() delegates to executionService.execute() with correct parameters")
void requestDelegatesToExecutionService() {
    AbstractParameters params = mock(AbstractParameters.class);
    TaskExecutionRequest request = new TaskExecutionRequest("merge", params);

    controller.request(request);

    verify(mockExecutionService, timeout(1000)).execute(params);
}

@Test
@DisplayName("request() passes different parameters to execute() for each request")
void requestPassesDifferentParams() {
    AbstractParameters params1 = mock(AbstractParameters.class);
    AbstractParameters params2 = mock(AbstractParameters.class);

    controller.request(new TaskExecutionRequest("merge", params1));
    controller.request(new TaskExecutionRequest("split", params2));

    // ArgumentCaptor captures actual arguments passed to execute()
    ArgumentCaptor<AbstractParameters> captor = ArgumentCaptor.forClass(AbstractParameter
    verify(mockExecutionService, timeout(1000).times(2)).execute(captor.capture());

    List<AbstractParameters> capturedParams = captor.getAllValues();
    assertEquals(2, capturedParams.size());
    assertSame(params1, capturedParams.get(0));
    assertSame(params2, capturedParams.get(1));
}

// NOTE: This test intentionally causes a RuntimeException inside the executor thread.
// The exception stack trace printed to stderr is EXPECTED and does not indicate a test fa
@Test
@DisplayName("execute() throwing exception does not prevent subsequent requests")
void executionServiceExceptionDoesNotBlock() {
    AbstractParameters failParams = mock(AbstractParameters.class);
    AbstractParameters successParams = mock(AbstractParameters.class);

    // Configure first call to throw, simulating a task failure
    doThrow(new RuntimeException("Task failed")).when(mockExecutionService).execute(failPa

    controller.request(new TaskExecutionRequest("merge", failParams));
```

```java
    controller.request(new TaskExecutionRequest("split", successParams));

    // Both calls should still be attempted (single-thread executor runs sequentially)
    verify(mockExecutionService, timeout(1000)).execute(failParams);
    verify(mockExecutionService, timeout(1000)).execute(successParams);
}

@Test
@DisplayName("request() calls execute() exactly once per request, no extra invocations")
void executeCalledOncePerRequest() {
    AbstractParameters params = mock(AbstractParameters.class);

    controller.request(new TaskExecutionRequest("rotate", params));

    verify(mockExecutionService, timeout(1000).times(1)).execute(params);
    verifyNoMoreInteractions(mockExecutionService);
}
```

🔬 11. Kingson's Mocking

Test File: KingsonPart5Test.java

11.1 Feature Mocked

`DefaultRecentWorkspacesService` — manages recently used workspaces with a `PreferencesRepository` for persistence.

Why mocking is needed: The existing tests in `DefaultRecentWorkspacesServiceTest` use a *real* `PreferencesRepository`, which writes to actual Java Preferences. With Mockito, we can:

- Verify the constructor properly reads from the repository
- Verify `onShutdown()` persists data in the correct order (clean first, then save)
- Verify `clear()` delegates to `repo.clean()`

11.2 Mock Test Cases

```
@Test
void constructorPopulatesCache() {
    when(mockRepo.keys()).thenReturn(new String[]{"k1"});
    when(mockRepo.getString("k1", "")).thenReturn("/path/to/workspace.json");

    var service = new DefaultRecentWorkspacesService(mockRepo);

    verify(mockRepo).keys();                  // Behavior: keys() was called
    verify(mockRepo).getString("k1", "");     // Behavior: each key was read
    assertEquals(1, service.getRecentlyUsedWorkspaces().size());
}

@Test
void onShutdownPersists() {
    var service = new DefaultRecentWorkspacesService(mockRepo);
    service.addWorkspaceLastUsed(new File("/path/to/workspace1.json"));

    service.onShutdown(new ShutdownEvent());

    verify(mockRepo, times(1)).clean();  // Clean before saving
    verify(mockRepo).saveString(anyString(), eq("/path/to/workspace1.json"));
}
```

🔬 12. Zian's Mocking

Test File: ZianPart5Test.java

12.1 Feature Mocked

`StageServiceController` — handles events related to window stage status by delegating to `StageService`.

Why mocking is needed: The controller receives events and forwards them to the service. Without mocking, we'd need a real `DefaultStageService` with a `DefaultEntityRepository`, which requires Java Preferences. Mocking lets us verify:

- Events are correctly forwarded to `service.save()`
- Status values are passed through without modification
- `CleanupRequest` triggers `service.clear()`

12.2 Mock Test Cases

```
@Test
void requestForwardsToSave() {
    SetLatestStageStatusRequest event = new SetLatestStageStatusRequest(StageStatus.NULL)
    controller.requestStageStatus(event);

    verify(mockService).save(StageStatus.NULL);  // Exact status forwarded
}

@Test
void requestPassesCorrectStatus() {
    StageStatus status = new StageStatus(100, 200, 800, 600);
    controller.requestStageStatus(new SetLatestStageStatusRequest(status));

    verify(mockService).save(status);
    verifyNoMoreInteractions(mockService);  // Only save, nothing else
}

@Test
void broadcastCleanupTriggersClear() {
    eventStudio().broadcast(new CleanupRequest());  // Via event bus

    verify(mockService).clear();  // Behavior: clear() was called
}
```

📋 13. Test Implementation Summary

13.1 New Test Files

File	Location	Author
ZhenyuPart5Test.java	`pdfsam-service/src/test/java/org/pdfsam/service/`	Zhenyu Song
KingsonPart5Test.java	`pdfsam-service/src/test/java/org/pdfsam/service/`	Kingson Zhang
ZianPart5Test.java	`pdfsam-service/src/test/java/org/pdfsam/service/`	Zian Xu

13.2 Test Breakdown

Member	Stubbing Tests	Bad Design Tests	Mocking Tests	Total
Zhenyu Song	3	5	4	**12**
Kingson Zhang	3	3	3	**9**
Zian Xu	3	4	5	**12**
Total	9	12	12	**33**

13.3 Running the Tests

```
# Run all Part 5 tests together
mvn test -pl pdfsam-service -Dtest="ZhenyuPart5Test,KingsonPart5Test,ZianPart5Test"

# Run individual test files
mvn test -pl pdfsam-service -Dtest=ZhenyuPart5Test
mvn test -pl pdfsam-service -Dtest=KingsonPart5Test
mvn test -pl pdfsam-service -Dtest=ZianPart5Test
```

13.4 Test Results

```
$ mvn test -pl pdfsam-service -Dtest="ZhenyuPart5Test"

[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.845 s -- in Mock
[INFO] Running Bad Testable Design Fix
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.065 s -- in Bad
[INFO] Running StubbingTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in Stubl
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.936 s -- in org.
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
```

```
$ mvn test -pl pdfsam-service -Dtest="KingsonPart5Test"

[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.027 s -- in Mock
[INFO] Running Bad Testable Design Fix: Injectable ExecutorService
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.222 s -- in Bad
[INFO] Running Stubbing: StubNewsService with NewsChecker
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in Stubl
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.303 s -- in org.
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
```

```
$ mvn test -pl pdfsam-service -Dtest="ZianPart5Test"

[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.445 s -- in Mock
[INFO] Running Bad Testable Design Fix: Injectable VersionDataProvider
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.618 s -- in Bad
[INFO] Running Stubbing: StubUpdateService with UpdateChecker
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s -- in Stub
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.117 s -- in org.
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
```

🎯 14. Conclusion

This report demonstrates the application of **testable design principles**, **stubbing**, and **mocking** to PDFsam Basic:

Section	Member	Technique	Target
Stubbing	Zhenyu	Manual `StubUsageService`	`TaskExecutionController`
Stubbing	Kingson	Manual `StubNewsService`	News checking logic
Stubbing	Zian	Manual `StubUpdateService`	Update checking logic
Bad Design	Zhenyu	Fix `private static` method	`DefaultPdfLoadService.fxMoveStatusTo()`
Bad Design	Kingson	Fix hardcoded `ExecutorService`	`TaskExecutionController`
Bad Design	Zian	Fix hardcoded URL creation	`DefaultUpdateService.getLatestVersion()`
Mocking	Zhenyu	Mockito verify `TaskExecutionService`	`TaskExecutionController.request()`
Mocking	Kingson	Mockito verify `PreferencesRepository`	`DefaultRecentWorkspacesService`
Mocking	Zian	Mockito verify `StageService`	`StageServiceController`

Key Takeaways

- **Interface-based design** in PDFsam enables easy stubbing and mocking
- **Dependency injection** allows test doubles to replace real dependencies
- **Bad testable design** patterns (static methods, hardcoded objects, embedded I/O) can be fixed by extracting dependencies into injectable interfaces
- **Mockito mocking** provides **behavior verification** that cannot be achieved with state-based assertions alone